

PatchPilot ViT

Vision Transformer Implementation Report

Repository name: patchpilot-vit

Description: A lightweight PyTorch Vision Transformer notebook demonstrating patch embeddings, transformer encoding, and an image-classification training loop.

Notebook	Vit.ipynb
Total cells	9
Code cells	9
Saved outputs	0
Dataset in notebook	Synthetic FakeCIFAR10

1. Project Goal

This notebook is an educational PyTorch implementation of a Vision Transformer for image classification. Its main purpose is to show how an image can be converted into a sequence of patch tokens and processed with the same attention-based mechanism used in Transformer models.

The notebook is currently architecture-focused rather than result-focused. It contains no saved execution outputs, plots, or final metric tables, and it uses random synthetic data instead of a real image dataset.

2. Dataset Pipeline

The notebook defines a custom Dataset class called FakeCIFAR10. It creates random tensors with shape [N, 3, 224, 224] and random integer labels from 0 to 9.

The training split contains 800 samples and the validation split contains 200 samples. DataLoader objects are then created with batch size 32.

Because the images and labels are random, the model cannot learn real visual patterns. Any accuracy reported after running this exact notebook should be treated as a sanity-check value, not as a meaningful classification result.

3. Vision Transformer Flow

Step	Component	Purpose
1	Input image	Receives a 224 x 224 RGB image.
2	PatchEmbedding	Uses Conv2d with kernel=stride=16 to create 196 patch embeddings.
3	Class token	Prepends a learnable token that summarizes the image for classification.
4	Position embedding	Adds learnable position information to preserve patch order.
5	Transformer encoder	Applies multi-head self-attention and feed-forward layers.
6	LayerNorm + linear head	Uses the class token to produce logits for 10 classes.

4. Model Configuration

The implementation uses image size 224, patch size 16, embedding dimension 256, six Transformer encoder layers, eight attention heads, MLP dimension 512, dropout 0.1, and ten output classes.

For a 224 x 224 image with 16 x 16 patches, the model creates $14 \times 14 = 196$ patch tokens. After adding the class token, the Transformer receives a sequence length of 197.

5. Training Method

The notebook uses CrossEntropyLoss, AdamW optimization with learning rate $3e-4$ and weight decay $1e-4$, and CosineAnnealingLR with $T_max=10$.

Only the classification head and final normalization layer are unfrozen in the current setup. The embedding block and Transformer encoder are frozen. This makes training lighter, but with random initialization and random data, it does not provide a meaningful transfer-learning setup.

The training loop stores the best state dictionary according to validation loss and reloads that best state after the final epoch.

6. Code-Level Notes

Two small issues should be fixed before running the notebook. First, a device variable should be defined before the model is moved to device: `torch.device('cuda' if torch.cuda.is_available() else 'cpu')`.

Second, the accuracy function is defined as `claculate_accuracy` but later called as `calculate_accuracy`. The function name should be corrected so training and validation can call it successfully.

For a repository version, it is also useful to save the best model to disk with `torch.save`, add real dataset loading, and plot loss/accuracy curves after training.

7. Result Interpretation

The notebook file inspected for this report has zero saved outputs. Therefore, this report does not claim final training accuracy, validation accuracy, or loss curves.

Even if the notebook is executed exactly as written after fixing the small errors, the reported accuracy will not measure real visual understanding because FakeCIFAR10 assigns random labels to random images.

A valid experimental result should be produced by replacing the synthetic dataset with a real dataset and then recording training loss, validation loss, training accuracy, validation accuracy, and confusion/error analysis.

8. Recommended Repository Scope

Keep the repository lightweight: include the notebook, this README, and the report. Avoid committing large datasets, model checkpoints, cache folders, or environment-specific notebook checkpoints.

A good first improvement is to create a second notebook section that replaces FakeCIFAR10 with `torchvision.datasets.CIFAR10` and records actual metrics.